



Java et la Programmation Orientée Objet



Houssem MAHMOUDI
houssem.mahmoudi@ensicaen.fr

Plan du module



- Chapitre 1 : Présentation du langage Java
- Chapitre 2 : La Programmation Orientée Objet
- Chapitre 3 : Gestion des exceptions
- Chapitre 4 : Les collections et les chaînes de caractères
- Chapitre 5 : Gestion des Flux, Fichiers et la Sérialisation
- Chapitre 6 : Les Threads (Programmation concurrentielle)
- Chapitre 7 : Outils en Java : Javadoc & archive JAR
- Chapitre 8 : Le graphisme en Java
- Chapitre 9 : Gestions des évènements
- Chapitre 10 : Java et la Base de données
- Chapitre 11 : Java et la sécurité

Chapitre 1 : Présentation du langage Java



1. Historique de Java
2. Définition
3. Caractéristiques du langage Java
4. Les différentes versions
5. Les environnements
6. Les outils de développements
7. Les IDE
8. Compilation et exécution d'un programme Java
9. Syntaxe de base
10. Structures de contrôles

1. Historique

- Java est un langage développé par Sun Microsystems depuis 1995 puis racheté par l'entreprise Oracle en 2010.
- Java a évolué à travers plusieurs versions majeures, chacune apporte des améliorations et augmentant le nombre de packages et de classes.

2. Définition

- Java est un langage de programmation orienté objet simple se caractérise par sa **portabilité**, sa robustesse et son indépendance vis-à-vis des plateformes.



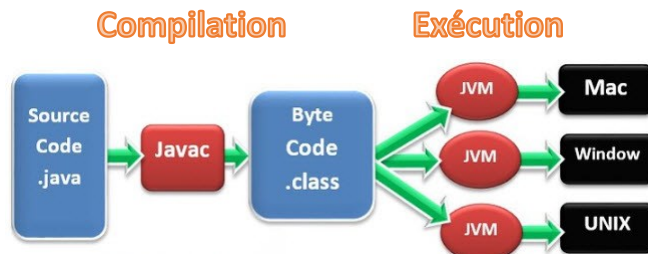
3. Caractéristiques du langage Java

Java possède des caractéristiques qui ont largement contribué à son succès :

- ✓ Java est orienté objet : Permet la modélisation des objets du monde réel par des classes. Chaque classe contient la déclaration des variables et la définition des méthodes.
- ✓ Java assure la gestion de la mémoire : Pas de manipulation du pointeur. L'allocation de la mémoire pour un nouveau objet est automatique à sa création. Java libère la mémoire inutilisée grâce au **Garbage Collector** (ramasse miette).
- ✓ Java est fortement typée : Toutes les variables sont typées. Si une conversion doit être réalisée, le développeur doit obligatoirement utiliser un **cast** ou une méthode pour l'effectuer.

3. Caractéristiques du langage Java

- ✓ Java est un langage compilé et interprété : Le fichier source (.java) est compilé par le compilateur **javac** en un pseudo code ou **bytecode** (.class) puis interprété en code machine par la Machine Virtuelle Java (JVM) au moment de l'exécution.



- ✓ Java est portable : Java est indépendant de toutes les plates-formes. On peut exécuter n'importe quelle classe sur n'importe quelle machine à condition qu'elle possède la Machine Virtuelle Java (JVM).

4. Différentes versions du langage Java

Le langage Java a connu de nombreuses versions depuis sa création :

- Java 1.0 : Première version publiée en 1996, a introduit les bases du langage Java.
- Java 1.1 : Publiée en 1997
- Java 1.2 ou Java 2 : Publiée en (1998)
- Java 1.3 (2000), Java 1.4 (2002), Java 1.5 ou Java 5 (2004), Java 6 (2006), Java 7 (2011), Java 8 (2014), Java 9 (2017), Java 10/11 (2018), Java 12/13 (2019), Java 14/15 (2020), Java 16/17 (2021), Java 18/19 (2022), Java 20/21 (2023)

4. Différentes versions du langage Java

Oracle offre des versions **LTS** (Long-Term Support) de Java qui sont caractérisées par un support prolongé, une stabilité à long terme et de mise à jour de sécurité régulières :

- Java 8 (2014) : Support LTS jusqu'à 2025
- Java 11 (2018) : Support LTS jusqu'à 2026
- Java 17 (2021) : Support LTS jusqu'à 2029
- Java 21 (2023) : Support LTS jusqu'à 2031

```
C:\Users\Houssem>java -version
java version "17.0.11" 2024-04-16 LTS
Java(TM) SE Runtime Environment (build 17.0.11+7-LTS-207)
Java HotSpot(TM) 64-Bit Server VM (build 17.0.11+7-LTS-207,
```

5. Les environnements Java

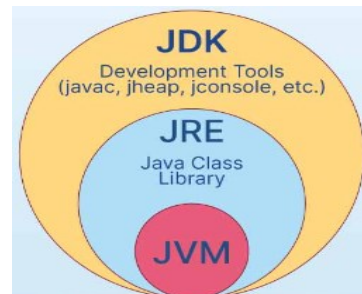
Java se présente sous différents éditions et bibliothèques utilisés pour le développement, l'exécution et la gestion des applications Java.

- Java Standard Edition (**Java SE**) : Met à la disposition du développeur les bibliothèques de bases pour créer des applications de bureau, Applet.
- Java Entreprise Edition (**Java EE**) : connu sous le nom Jakarta EE, permet de créer des applications d'entreprise robustes comme des applications Web, des services Web, EJB et la gestion de transaction.
- Java Micro Edition (**Java ME**) : Environnement conçu pour le développement mobiles et embarqués.



6. Les outils de développements

- Java Development Kit (**JDK**) : Comprends les outils nécessaires de développements des applications Java (compilateur : **javac**, interpréteur d'application : **java**, débogueur : **jdb**, générateur de documentation : **javadoc**) et les bibliothèques de base du langage Java. Il contient aussi le **JRE**.
- Java Runtime Environment (**JRE**) : Comprends la Machine Virtuelle Java (**JVM**) et les bibliothèques nécessaires à l'exécution des applications Java.



7. Les environnements de développement intégrés (IDE)

- **Eclipse** : Un IDE open-source largement utilisé pour le développement Java, avec une multitude de plugins téléchargeable pour étendre ses fonctionnalités.
 - Site : <https://www.eclipse.org/>
- **IntelliJ IDEA** : Un IDE commercial mais propose une version gratuite sous le nom de Community Edition. Caractérisé par sa puissante fonctionnalités de développement.
 - Site : <https://www.jetbrains.com/idea/>
- **Netbeans** : Un IDE open-source sponsorisé par Oracle, qui supporte Java SE, Java EE. Caractérisé par une interface utilisateur riche et supporte plusieurs langages.
 - Site : <https://netbeans.apache.org/>



8. Compilation et exécution d'un programme Java

- Soit le fichier Test.java

```
public class Test {
    public static void main(String args []) {
        System.out.println("\n Bonjour mes etudiants. ");
    }
}
```

- Pour compiler le fichier source dans l'invite de commandes : C:\> javac Test.java
- Si la compilation se passe sans erreurs, on obtient un fichier **Test.class**
- Pour exécuter le programme : C:\> java Test

8. Compilation et exécution d'un programme Java

Remarques :

- Le nom de la classe et le nom du fichier source doivent être identique.
- Par convention le nom de la classe commence par une lettre en majuscule.
- Si le fichier source contient plusieurs classes, après compilation on obtient autant de fichiers classes que déclarées dans le fichier source.
- Dans un fichier Java, une seule classe admet l'attribut public qui contient la méthode main.
- La méthode **main()** est le point d'entrée du programme Java. C'est là où commence l'exécution du programme.
- Un fichier Java peut ne pas contenir une classe publique, ni une méthode main. Dans ce cas on ne peut pas l'exécuter. Il sert à être instancié ou hérité.

9. Syntaxe de base

a. Les variables

- Java possède trois types de variables :
 - Les **variables d'instances** : Déclarées dans la classe et en dehors des méthodes.
 - Les **variables locales** : Déclarées au sein des méthodes.
 - Les **variables de classe** : Variables partagées par toutes les objets (voir plus loin).
- Java est un langage fortement typé, cela signifie que le type de chaque variable doit être déclaré avant d'être utilisée.

b. Les constantes

Java permet de déclarer des constantes en utilisant le mot clé **final**, sa valeur ne doit pas être modifiée pendant l'exécution du programme.

Exemple :

1	final int n = 20 ;	Initialisation directe.
2	final int n ; n = 10 ;	Java autorise qu'une variable déclarée final ne reçoive qu'une seule valeur.

9. Syntaxe de base

b. Les Types Numériques

i. byte :

- Taille : 8 bits
- Plage de valeurs : [-128 .. 127]
- Exemple : byte b = 10;

ii. short :

- Taille : 16 bits
- Plage de valeurs : [-32,768 .. 32,767]
- Exemple : short s : 100;

iii. int :

- Taille : 32 bits
- Plage de valeurs : $-2^{31} .. 2^{31} - 1$
- Exemple : int i = 1000;

9. Syntaxe de base

b. Les Types Numériques

iv. long :

- Taille : 64 bits
- Plage de valeurs : $-2^{63} \text{ à } 2^{63} - 1$
- Exemple : long l = 2000L;

v. float :

- Taille : 32 bits
- Plage de valeurs : $-3.4028235 \times 10^{38} .. 3.4028235 \times 10^{38}$
- Exemple : float f = 3.14F;

⚙ float g = 3.14; erreur : Type mismatch
 ➔ impossible de convertir de double en float

vi. double :

- Taille : 64 bits
- Plage de valeurs : $-1.7976931 \times 10^{308} .. 1.7976931 \times 10^{308}$
- Exemple : double d = 5.75;

9. Syntaxe de base

c. Type boolean

- Valeurs : *true* ou *false*.
- Un entier non nul est assimilé à *true*.
- Un entier nul est assimilé à *false*.

d. Type char

- Une variable de type `char` contient un seul caractère codé sur 16 bits
- Valeurs : 0 à 65535 (représentant un seul caractère Unicode)
- Exemple : `char c = 'A'`

9. Syntaxe de base

e. Type String

- Les chaînes de caractères en Java sont manipulées par la classe **String**.
- Les objets de la classe `String` ne sont pas modifiables.
 - `String s = "Hello World";`
 - `s[2] = "p";` provoque une erreur de compilation !
- La classe `String` fournit des méthodes pour manipuler les chaînes de caractères :
 - **length()**:int : renvoie la longueur de la chaîne
 - Exemple : `s.length();` → renvoie 11
 - **concat(String):String** : concaténer l'objet en cours avec la chaîne passer en paramètre
 - Exemple : `String t = " Code";`
`s=s.concat(t);`
`s` contient la chaîne : Hello World Code

9. Syntaxe de base

e. Type String

- **charAt(int):char** : retourne un caractère à partir de sa position
 - Exemple : `s.charAt(1);` → renvoie ``e``
- **equals(String):boolean** : Comparer s avec la chaîne passée en paramètre
 - Exemple : `boolean b = s.equals("Hello World");` → Renvoie `true`
- **equalsIgnoreCase(String):boolean** : Réaliser la comparaison en ignorant la casse.
 - Exemple : `boolean b = s.equals("Hello World");` → Renvoie `true`
- **substring(int, int):String** : Extraire une sous chaîne du premier index inclusive au dernier index exclusive.
 - Exemple : `String m = s.substring(0,5);` → Renvoie `Hello`

9. Syntaxe de base

e. Type String

- **split(String):String[]** : diviser la chaîne en se basant sur un séparateur et retourner un tableau de chaînes de caractères.
 - Exemple : `s.split(" ");` → diviser la chaîne en se basant sur le séparateur espace; Retourne `["Hello", "World"]`
- **replace(char, char):String** : Remplace toute occurrence du 1^{er} caractère par le 2^{ème}.
 - Exemple : `String x = s.replace('l', 'x');` → Renvoie la chaîne `Hexxo worxd`
- **toLowerCase()** : String : retourne la chaîne en minuscule.
 - Exemple : `s = s.toLowerCase();`
- **toUpperCase()** : String : retourne la chaîne en majuscule
 - Exemple : `s = s.toUpperCase();`

9. Syntaxe de base

e. Les opérateurs arithmétiques

Les opérateurs arithmétiques sont les suivants : l'addition (+), la soustraction (-), la multiplication (*), la division (/) et l'opérateur modulo (%).

f. Les opérateurs relationnels

Voici la liste des opérateurs relationnels existant en Java

Opérateur	Signification
<	Inférieur à
<=	Inférieur ou égal à
>	Supérieur à
>=	Supérieur ou égal à
==	Egal à
!=	Différent de

9. Syntaxe de base

g. Les opérateurs logiques

Java dispose d'opérateurs logiques suivants :

Opérateur	Signification
!	Négation
&	Et
^	Ou exclusif
	Ou inclusif
&&	Et (avec court-circuit)
	Ou inclusif (avec court-circuit)

9. Syntaxe de base

h. L'opérateur d'affectation

- L'opérateur d'affectation dans le langage Java est le signe (=), il permet d'affecter une valeur ou, le résultat d'une expression, dans une variable.
- On peut utiliser l'opérateur d'affectation élargie qui consiste à remplacer l'écriture $i = i + k$ par $i += k$ ou encore $a = a * b$ par $a *= b$
- Attention à l'affectation des types différents :

Exemple 1 : int a ; float x ; x = a + 5 ;	On demande au compilateur d'affecter une valeur de type int à une variable de type float, Java l'accepte et réalise une <u>conversion implicite</u>
Exemple 2 : int n ; float x ; n = x + 5 ;	Cette affectation sera rejetée par le compilateur, Java refuse de convertir une valeur de type float en int implicitement. Il faut réaliser une <u>conversion explicite</u> ou forcée (casting) : $n = (int) x + 5 ;$

9. Syntaxe de base

h. Conversion de types

- Voici le sens des conversions implicites réalisées par Java :
 - byte → short → int → long → float → double
 - char → int → long → float → double

Remarque :

- Le type de résultat d'une expression comportant byte ou short est toujours un int.

Exemple :

```
byte a = 10 ; byte b = 7 ;
```

```
byte c = a + b ;      ➔ ici il y a une faute de compilation,
```

- En Java, toute opération arithmétique (+, -, *, /) sur des types plus petits que int est automatiquement convertie en int.
- Le type de résultat de la somme des deux *byte* est un *int*, il faut faire un casting (conversion explicite) vers byte. ➔ `byte c = (byte) (a + b) ;`

10. Les structures de contrôles

a. L'instruction if

```
if (conditions) {
    instructions 1 ;
}
else {
    instructions 2 ;
}
```

```
if (conditions1) {
    instructions_1 ;
}
else if (conditions2) {
    instructions_2 ;
}
else {
    instructions_3 ;
}
```

Remarque :

En Java, il est possible de traduire cette instruction « Si (a>b) alors a sinon b » par une opération conditionnelle ternaire : `max = a>b ? a : b ;`

9. Les structures de contrôles

b. L'instruction switch

```
switch (expression) {
    case constante_1 : instructions ;
                        break ;
    case constante_2 : instructions ;
                        break ;
    ... ..
    default : instruction par défaut ;
}
```

Remarques :

- Constante_i est une constante de type compatible avec le type d'expression.
- Chaque instruction doit terminer avec le mot clé `break`, sinon on exécute le reste du bloc.
- La partie **default** est facultative.

9. Les structures de contrôles

c. L'instruction `do .. while`

```
do
{
    instructions ;
}
while (conditions) ;
```

- La boucle `do .. while` permet de répéter un bloc d'instruction, elle s'exécute au moins une fois, même si la condition est fausse.
- La boucle répète les instructions tant que la condition est vraie.
- Il est possible de sortir du boucle `do .. while` en utilisant le mot clé ***break***;

9. Les structures de contrôles

d. L'instruction `while`

```
while (conditions)
{
    instructions ;
}
```

- La boucle `while` répète les instructions si la condition mentionnée est vraie.
- La condition est examinée à chaque fois avant d'exécuter le bloc.
- Il est important d'initialiser la valeur de la condition avant d'exécuter la boucle `while`.

9. Les structures de contrôles

e. L'instruction for

```
for (initialisation ; conditions ; incrémentation)
{
    instructions ;
}
```

Démarche d'exécution de la boucle **for** :

- La partie initialisation sera évaluée une seule fois avant d'entrer dans la boucle.
- La partie condition conditionne la poursuite de la boucle, elle est évaluée avant chaque parcours.
- La partie incrémentation est évaluée à la fin de chaque parcours.

9. Les structures de contrôles

f. Branchement inconditionnel : break et continue

Break : permet d'interrompre le déroulement d'une boucle

Exemple : soit le code Java suivant

```
for (int i = 1 ; i < 5 ; i++) {
    System.out.println ("début numéro : " + i) ;
    if(i==3) break ;
    System.out.println ("fin numéro : " + i) ;
}
```

Résultat :

```
début numéro : 1
fin numéro : 1
début numéro : 2
fin numéro : 2
début numéro : 3
```

9. Les structures de contrôles

f. Branchement inconditionnel : break et continue

Continue : permet de passer au tour suivant de la boucle.

Exemple : soit le code Java suivant

```
for (int i = 1 ; i < 5 ; i++)  
{ System.out.println ("début numéro : " + i) ;  
  if(i<4) continue ;  
  System.out.println ("fin numéro : " + i) ;  
}
```

Résultat :

```
début numéro : 1  
début numéro : 2  
début numéro : 3  
début numéro : 4  
fin numéro : 4  
début numéro : 5  
fin numéro : 5
```